MODULE 8

AI-Driven Quality Assurance

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Eighth**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Use Claude Code to design comprehensive test suites with targeted coverage prompts and systematic edge case generation
2. Implement pytest fixtures, mocks, and test isolation patterns generated by Claude Code for reliable and maintainable tests
3. Diagnose and fix flaky tests using Claude Code's analysis of test output, timing patterns, and state dependencies
4. Understand property-based testing concepts and how to prompt Claude Code to generate hypothesis-based tests
5. Apply mutation testing concepts to evaluate test suite quality and identify gaps in assertion coverage
6. Choose the right testing strategy for different scenarios: unit vs. integration, mocks vs. real dependencies, speed vs. thoroughness

SESSION TIMING BREAKDOWN

0:00 – 0:10	**Section 1: AI-Driven Test Design — Coverage & Edge Cases**
0:10 – 0:20	**Section 2: Fixtures, Mocks & Test Isolation Patterns**
0:20 – 0:28	**Section 3: Flaky Test Triage with Claude Code**
0:28 – 0:37	**Section 4: Property-Based Testing & Mutation Testing**
0:37 – 0:43	**Section 5: Testing Strategy Decision Framework**
0:43 – 0:45	**Wrap-Up: Quick Reference & Day 1 Close**

0:00 – 0:10	SECTION 1: AI-Driven Test Design — Coverage & Edge Cases

1. AI-Driven Test Design: Coverage Prompts & Edge Case Generation

1.1 The Coverage-First Approach

In Module 6 you generated basic tests. Now we'll use Claude Code strategically — not just to write tests, but to design test suites that systematically cover all paths, boundaries, and failure modes. The key is prompting Claude with coverage targets, not just "write tests."

Basic Test Prompt	Coverage-Driven Test Prompt
"Write tests for validators.py"	"Generate tests for validate_reading() targeting 100% branch coverage. Include: every if/else path, boundary values for each range (exactly at min, max, one below, one above), and all three sensor_id format violations."
Claude decides what to test	You specify the coverage strategy
Random selection of cases	Systematic: paths → boundaries → errors
Might miss edge cases	Explicitly targets edge cases

1.2 Systematic Edge Case Generation

Ask Claude to think through edge cases methodically using the BICEP framework:

BICEP Category	Example Tests for validate_reading()
Boundary	temp at exactly -40 and 150 (inclusive boundaries), pressure at 0 and 1000
Inverse	Valid reading returns empty list, invalid returns non-empty list (verify both directions)
Cross-check	validate_batch result totals match: valid + invalid == total
Error	None values, wrong types (str instead of float), missing fields, empty string sensor_id
Performance	validate_batch with 10,000 readings completes within 1 second

```
# Generate edge cases systematically:
> For @src/sensor_toolkit/validators.py, generate pytest tests
> using the BICEP framework:
> - Boundary: all range limits for temp, pressure, humidity
> - Inverse: valid input gives no errors, invalid gives errors
> - Cross-check: batch totals are internally consistent
> - Error: None, wrong types, missing fields, empty strings
> - Performance: batch of 10K readings under 1 second
> Use @pytest.mark.parametrize for boundary values.
> Group tests in classes by category.
```

> ☆ **KEY POINT:** The BICEP framework gives Claude a structured approach to edge case generation. Without it, Claude tends to generate happy-path tests plus a few obvious failures. With BICEP, you get systematic coverage that finds real bugs.

1.3 Coverage Gap Analysis

```
# Run coverage, then ask Claude to fill gaps:
> !pytest --cov=src/sensor_toolkit --cov-report=term-missing

# Paste the missing lines to Claude:
> Coverage report shows lines 34-38 and 52-55 in validators.py
> are not covered. These are the error handling paths for
> invalid sensor_id format and None values. Generate tests
> that exercise exactly these lines.
```

0:10 – 0:20	SECTION 2: Fixtures, Mocks & Test Isolation Patterns

2. Fixtures, Mocks & Test Isolation Patterns

2.1 Fixture Architecture

Well-structured fixtures are the backbone of maintainable test suites. Claude Code can generate fixture hierarchies, but you need to tell it how your team organizes them.

```
# Ask Claude to build a fixture architecture:
> Create a comprehensive conftest.py for the sensor_toolkit
> test suite with the following fixture hierarchy:
>
> Session-scoped fixtures:
>   - sample_schema: validation schema dict (created once)
>
> Module-scoped fixtures:
>   - sensor_readings_csv: temp file with 100 valid readings
>
> Function-scoped fixtures (default):
>   - valid_reading: single valid SensorReading
>   - invalid_reading: SensorReading with temp=999
>   - reading_batch: list of 10 mixed valid/invalid readings
>   - empty_batch: empty list
>
> Fixture factories:
>   - make_reading(**overrides): creates SensorReading with
>     defaults, any field overridable via kwargs
```

☆ **KEY POINT:** Fixture factories are the most powerful testing pattern. make_reading(temperature=200) creates a reading that's valid in every way except temperature. This lets you test one validation rule at a time without constructing entire objects manually.

2.2 Mocking External Dependencies

When your code reads files, calls APIs, or queries databases, you need mocks to isolate the logic you're testing:

```
# Ask Claude to add mocks for file I/O:
> Generate tests for validate_csv_file() that mock
> file I/O using pytest's tmp_path fixture and
> unittest.mock.patch. Include:
> - Test with a valid CSV (write test CSV to tmp_path)
> - Test with malformed CSV (missing columns)
> - Test with empty CSV file
> - Test with file not found (mock open to raise FileNotFoundError)
> Do NOT actually read from the filesystem in unit tests.

# Ask Claude to mock an API dependency:
> Generate tests for fetch_sensor_data() that mock the
> requests.get call. Test: successful response (200),
> timeout error, and JSON decode error.
> Use unittest.mock.patch as a decorator.
```

Track A: Advanced Mocking	Track B: Simple Mocking
Mock complex dependencies: databases, APIs, message queues. Use MagicMock with spec=True for type safety.	Mock file I/O with tmp_path. Use simple mock.patch for one dependency at a time.
Mock chains: request → response → json(). Use side_effect for conditional returns.	Focus on "when this fails, test handles it." Use side_effect=Exception for error cases.
Discuss mock.AsyncMock for async code, context managers.	Keep mocks simple. If mocking feels hard, refactor the code (Module 7 callback).

⚠ **WARNING:** Over-mocking is a real risk. If you mock everything, your tests prove nothing about how the code actually works. Rule of thumb: mock at the boundary (file I/O, network, database), but test real logic with real data. Claude will mock aggressively if you don't constrain it.

0:20 – 0:28	SECTION 3: Flaky Test Triage with Claude Code

3. Flaky Test Triage with Claude Code

3.1 What Makes Tests Flaky?

A flaky test passes sometimes and fails sometimes with no code changes. Flaky tests erode team trust in the test suite — developers start ignoring failures, and real bugs slip through. Claude Code can analyze flaky tests and diagnose root causes.

Flaky Pattern	Root Cause	Claude Code Fix
Passes alone, fails in suite	Shared state between tests (global variable, database row, temp file)	Ask Claude to add proper setup/teardown and test isolation
Fails intermittently	Timing dependency (sleep, async race, network timeout)	Ask Claude to replace time.sleep with polling/retry patterns
Fails on CI, passes locally	Environment difference (path, timezone, locale, Python version)	Ask Claude to add environment-agnostic assertions
Fails after 5 PM	Timezone or date-dependent logic (hardcoded dates, DST)	Ask Claude to mock datetime.now() with freezegun
Order-dependent failures	Test B depends on side effect from Test A	Ask Claude to make each test fully independent

3.2 Diagnosing Flaky Tests with Claude Code

```
# Step 1: Run the test multiple times to confirm flakiness:
> !pytest tests/test_analyzers.py::test_anomaly_detection
>     --count=10 -v  # (requires pytest-repeat plugin)

# Step 2: Paste the mixed pass/fail output to Claude:
> This test passes 7/10 times. Here are the 3 failure
> outputs: [paste tracebacks]. The test checks anomaly
> detection on sensor data. Why is it flaky?

# Step 3: Claude analyzes patterns:
#   - Checks if test uses random data (no seed)
#   - Checks for float comparison issues (0.1 + 0.2 != 0.3)
#   - Checks for timing dependencies
#   - Checks for shared state across tests

# Step 4: Apply the fix and re-run 10x to confirm:
> Run the test 10 times to verify the fix is stable
```

3.3 Common Fixes Claude Code Applies

```
# Fix 1: Replace exact float comparison with approx
# Before (flaky):
assert result.mean_temp == 25.333333
# After (stable):
assert result.mean_temp == pytest.approx(25.333, rel=1e-3)
```

```
# Fix 2: Mock time for date-dependent tests
# Before (flaky after 5 PM):
def test_daily_report():
    report = generate_report()  # uses datetime.now()
# After (stable, frozen time):
from freezegun import freeze_time
@freeze_time('2025-06-15 10:00:00')
def test_daily_report():
    report = generate_report()  # always 10 AM June 15

# Fix 3: Add random seed for reproducible tests
# Before (flaky with random test data):
data = [random.uniform(-50, 200) for _ in range(100)]
# After (reproducible):
rng = random.Random(42)  # fixed seed
data = [rng.uniform(-50, 200) for _ in range(100)]
```

☆ **KEY POINT:** When Claude diagnoses a flaky test, it's not just fixing the symptom — it's identifying a test design flaw. Every flaky test fix teaches you a pattern: use pytest.approx for floats, freeze time for date logic, seed randomness, isolate state. These patterns prevent future flakiness.

0:28 – 0:37	SECTION 4: Property-Based Testing & Mutation Testing

4. Property-Based Testing & Mutation Testing Concepts

4.1 Property-Based Testing: Testing Rules, Not Examples

Traditional tests check specific examples: “validate_reading with temp=200 should return an error.” Property-based testing checks rules: “For ANY temperature above 150, validate_reading should always return an error.” Python’s Hypothesis library generates hundreds of random test cases to verify the rule holds.

Example-Based Test	Property-Based Test
Tests one case: temp=200	Tests thousands: temp=151..inf (random)
You choose the inputs	Hypothesis generates inputs
Finds bugs you anticipated	Finds bugs you didn’t think of
def test_high_temp(): assert ...	@given(st.floats(min=151)) def test_high_temp(temp): assert ...
Fast, deterministic	Slower, randomized (but reproducible with seed)

```
# Ask Claude to generate property-based tests:
> Generate Hypothesis property-based tests for
> validate_reading() in @src/sensor_toolkit/validators.py.
> Test these properties:
>
> 1. Any reading with all values in valid ranges returns
>    an empty error list
> 2. Any reading with temp outside [-40, 150] returns
>    a list containing a temperature error
> 3. validate_batch always returns total == valid + invalid
> 4. validate_batch([]) returns {total:0, valid:0, invalid:0}
>
> Use hypothesis.strategies for generating values:
>   floats(min_value=-40, max_value=150) for valid temp
>   floats(min_value=151) for invalid temp
```

☆ **KEY POINT:** Property-based tests are Claude Code’s superpower for testing. You describe the property in natural language, Claude translates it into Hypothesis strategies. The generated tests find edge cases you’d never think to write manually — like NaN, infinity, or extremely small floats.

4.2 Mutation Testing: How Good Are Your Tests?

Mutation testing answers: “If I introduce a bug, do my tests catch it?” A mutation testing tool changes your code (e.g., replaces > with >=) and re-runs your tests. If tests still pass, they’re not catching that bug category.

Mutation	Code Change	Tests Should
Boundary	Change > to >=	Fail: boundary test catches it
Negation	Change if valid to if not valid	Fail: logic inverted

Value	Change -40 to -39	Fail: boundary shifted
Remove	Delete the pressure check entirely	Fail: missing validation
Return	Change return errors to return []	Fail: always returns empty

```
# Ask Claude to think like a mutation tester:
> Look at @src/sensor_toolkit/validators.py and identify
> 5 mutations that would change behavior but might NOT be
> caught by the current tests in @tests/test_validators.py.
> For each mutation, tell me:
> 1. What you'd change (the mutation)
> 2. Why current tests might miss it
> 3. What test to add to catch it

# Claude analyzes gaps between code logic and test assertions
# and generates targeted tests for each gap.
```

Track A: Advanced	Track B: Practical
Install mutmut and run actual mutation testing on validators.py. Analyze surviving mutants.	Use Claude's manual mutation analysis (prompt above). No extra tools needed.
Generate Hypothesis property-based tests for all three modules.	Generate 2-3 parametrized tests that cover boundary values you missed.
Discuss: when property tests find bugs that example tests miss (NaN, inf, empty string).	Focus on: what's the one test that would catch the most bugs? Claude helps prioritize.

0:37 – 0:43	SECTION 5: Testing Strategy Decision Framework

5. Testing Strategy Decision Framework

5.1 Choosing the Right Test Type

Not every function needs the same testing approach. Use this decision framework to choose the right strategy for each scenario:

Question	If Yes	If No
Does it have pure logic (no I/O)?	Unit test with real data	Consider mocking or integration test
Does it read files or call APIs?	Mock at the boundary, test logic separately	Unit test directly
Is it a critical business rule?	Property-based + example tests + mutation analysis	Example-based tests are sufficient
Does it handle money or safety?	100% branch coverage, all boundary values, negative tests	80% coverage is acceptable
Is it a simple utility function?	3-5 focused tests (happy, edge, error)	More comprehensive suite needed
Does it coordinate other functions?	Integration test with real (not mocked) dependencies	Unit test in isolation

5.2 The Testing Pyramid for TI Projects

Apply the testing pyramid to your sensor toolkit project:

Test Level	What to Test in Sensor Toolkit
Unit Tests (70%)	Individual validators, cleaners, analyzers. Pure functions with real data. Fast, isolated, many.
Integration Tests (20%)	CSV file reading + validation pipeline. validate_csv_file() with real temp files. Test the wiring.
End-to-End Tests (10%)	Full pipeline: read raw data → validate → clean → analyze → generate report. One or two happy-path tests.

5.3 Speed vs. Thoroughness Trade-offs

```
# Ask Claude to optimize test suite speed:
> Analyze the test suite in tests/. Identify tests that
> are slow (> 100ms) and suggest how to speed them up
> without losing coverage. Common fixes: reduce fixture
> scope, use lighter test data, mock slow I/O.

# Ask Claude to optimize thoroughness:
> Analyze the test suite in tests/. Identify the highest-risk
> functions that have the lowest coverage. Prioritize by:
> business impact (what breaks if this function is wrong?).
```

```
> Generate tests for the top 3 gaps.
```

☆ **KEY POINT:** In a TI production environment, the validators are safety-critical — they determine whether sensor data enters the pipeline. These need 100% branch coverage, property-based tests, and mutation analysis. The analyzers are informational — 80% coverage with example-based tests is sufficient. Allocate your testing effort based on business risk.

0:43 – 0:45	WRAP-UP: Quick Reference & Day 1 Close

6. Quick Reference & Day 1 Close

MODULE 8 RECAP

✅ Coverage-first test design: use coverage reports to identify gaps, then prompt Claude to generate targeted tests for missing lines and branches.

✅ BICEP framework: Boundary, Inverse, Cross-check, Error, Performance — systematic edge case generation that finds real bugs.

✅ Fixture architecture: session/module/function scopes, fixture factories (make_reading(**overrides)), conftest.py organization.

✅ Mocking strategy: mock at boundaries (file I/O, network, database), test real logic with real data. Avoid over-mocking.

✅ Flaky test diagnosis: shared state, timing dependencies, float precision, date/timezone logic. Fixes: isolation, pytest.approx, freezegun, random seeds.

✅ Property-based testing: describe rules, not examples. Hypothesis generates thousands of cases including edge cases you'd never think of.

✅ Mutation testing: "If I introduce a bug, do tests catch it?" Use Claude's manual mutation analysis to find test gaps.

✅ Testing pyramid: 70% unit (fast, many), 20% integration (real wiring), 10% E2E (full pipeline). Allocate effort by business risk.

QUICK REFERENCE CARD

Technique	Claude Code Prompt Pattern	When to Use
Coverage gaps	> pytest --cov shows lines X-Y missing. Generate tests for those lines.	After initial test suite
BICEP edge cases	> Generate tests using BICEP: Boundary, Inverse, Cross-check, Error, Performance.	Critical business logic
Fixture factory	> Create make_reading(**overrides) fixture factory in conftest.py.	Reusable test data
Mock file I/O	> Mock open() or use tmp_path for file-dependent tests.	Functions that read/write files
Fix flaky test	> This test passes 7/10 times. [tracebacks]. Why is it flaky?	Intermittent failures
Property-based	> Generate Hypothesis tests for property: [describe the rule].	Rules that hold for all inputs
Mutation analysis	> Identify 5 mutations current tests wouldn't catch. Suggest fixes.	Evaluate test quality
Speed optimization	> Find slow tests (>100ms) and suggest speedups.	CI pipeline too slow

DAY 1 CLOSING

Congratulations — you've completed Day 1! Here's what you've accomplished:

Module	What You Learned
M1: Big Picture & Safety	Why vibe coding matters, safety guardrails, hands-on installation
M2: Prompting Fundamentals	Clarity, context, constraints, iterative refinement, @-references
M3: Advanced Patterns	Problem decomposition, ReAct loop, temperature/thinking controls, CLAUDE.md
M4: CLI & VS Code Deep Dive	Interactive mode, slash commands, checkpoints, permissions, VS Code integration
M5: Automation & Scripting	Headless mode (-p), JSON output, piping, system prompts, automation recipes
M6: Lab 1 — Scaffold & Tests	Built a real project from scratch with Claude Code in 40 minutes
M7: Refactor & Debug	Refactoring prompts, testability, documentation, debugging, checkpoints
M8: Quality Assurance	Coverage-driven testing, fixtures/mocks, flaky tests, property/mutation testing

Tomorrow (Day 2): Custom Slash Commands, Sub-agents, MCP integrations, GitHub Actions CI/CD, and your capstone project.